

# Leksjon 1: KI-støttet applikasjonsutvikling for web

## 5. KI-støttet applikasjonsutvikling [arbeidskrav]

Bruk KI for å generere en prototype for landingssiden til [prosjektoppgaven i APP2000 for studieåret 2025/2026 \(turapp\)](#) . Sørg for at prototypen bygger kun med HTML og CSS, altså en statisk nettside uten backend/database. Prøv å eller flere av "KI-assistentene" lenket til under, og skriv en superkort oppsummering av forskjellige mellom løsningene du testet ut:

### Bruk av KI-verktøy til prototype (HTML/CSS)

Jeg testet flere KI-løsninger med denne prompten:

*«Generer en prototype for landingssiden til prosjektoppgaven i APP2000 for studieåret 2025/2026 (turapp), med kun HTML og CSS, statisk nettside uten backend/database.»*

Verktøyene jeg brukte var Replit, Sikt KI og GitHub Copilot.

I Replit fikk jeg generert en enkel landingsside som kunne kjøres direkte i nettleseren. Det gjorde det lett å se resultatet med en gang, og å teste og endre små ting i HTML og CSS fortløpende. Sikt KI var mest nyttig til tekstinnhold, for eksempel å formulere beskrivelser av turappen, turer, hytter og reservasjon på en litt mer ryddig måte. GitHub Copilot ga løpende forslag til både HTML- og CSS-kode i VS Code, og hjalp meg å få et mer strukturert og «profesjonelt» oppsett på selve koden.

I alle tilfellene endte jeg opp med det samme på et overordnet nivå: en prototype, altså en enkel, statisk HTML/CSS-side uten ekte funksjonalitet, backend eller database.

### Begreper knyttet til bruk av KI

I denne sammenhengen kan man bruke KI på to måter:

1. til å jobbe raskere og enklere når man utvikler, og
  2. til å lage funksjoner i selve webapplikasjonen som bruker KI.
- Under forklarer jeg kort noen sentrale begreper, med utgangspunkt i det.

#### Prompt

En prompt er teksten som skrives inn til KI for å få noe tilbake. For eksempel ble det brukt en prompt for å få generert landingssiden til turappen. Hvordan prompten formuleres har mye å si for hvor gode svar du får, både når du ber om kode, og når du bygger funksjoner der brukeren skal «snakke» med KI.

#### Token

Et token er en liten bit tekst som modellen jobber med. Modellen har en grense for hvor

mange tokens den kan håndtere om gangen. I praksis betyr det at det er en grense for hvor mye tekst/kode du kan sende inn og fortsatt få fornuftige svar, både i utvikling og i en ferdig app.

### **Copilot**

En Copilot er en KI som sitter «ved siden av» deg mens du koder og foreslår kode, slik GitHub Copilot gjør i VS Code. Det gjør at du slipper å skrive alt fra scratch, og du får ofte bedre struktur på koden. Samme idé kan også brukes for sluttbrukere, for eksempel en «Copilot» som hjelper dem å skrive tekst eller planlegge en tur.

### **Coding Agent**

En coding agent er en litt «smartere» KI som ikke bare foreslår én og én linje, men som kan planlegge og lage større deler av et system ut fra en beskrivelse eller spesifikasjon. Den kan for eksempel generere flere filer, sette opp struktur og lage funksjoner. Den samme typen agent kan også brukes inne i en app, for eksempel til å generere turforslag eller ruteplaner for brukeren.

### **Agent Skill**

En agent skill er en bestemt ting agenten kan gjøre, for eksempel «lese filer», «kalle et API», «lage HTML», eller «validere et skjema». I utvikling kan slike skills gjøre KI-verktøy mer nyttige. I en app kan du tenke på det som egne funksjoner, for eksempel «finn hytter som passer denne brukeren» eller «lag pakkelliste til fjellturen».

### **AI API**

Et AI API er en tjeneste på nett som du kan sende tekst eller data til fra koden din, og få et KI-svar tilbake. Dette er måten utviklere kobler KI inn i egne systemer og apper på. Hvis turappen for eksempel skulle hatt en chatassistent eller et anbefalingssystem, ville det typisk vært bygget på et AI API i bakgrunnen.

### **AI SDK**

Et AI SDK er et bibliotek (for eksempel for JavaScript) som gjør det enklere å bruke et AI API i koden. I stedet for å bygge alt fra bunnen av, får du ferdige funksjoner for å sende forespørsler og håndtere svar.

### **AI CLI**

En AI CLI er et kommandolinjeverktøy for å bruke KI direkte fra terminalen. Du kan få generert kode, forklart filer eller få forslag til endringer uten å åpne en egen editor. Dette er mest nyttig for utviklere, og ikke noe sluttbrukeren av en turapp ser.

### **Model Context Protocol (MCP)**

Model Context Protocol (MCP) er en måte å koble KI-modellen til forskjellige kilder på, som filer, API-er eller databaser, på en strukturert og kontrollert måte. Poenget er at modellen skal kunne forstå prosjektet eller appen bedre, uten at man roter med sikkerhet eller struktur. Dette er særlig viktig hvis man skal bygge mer avanserte KI-funksjoner som trenger tilgang til reelle data fra appen.

## **6. Spesifikasjonsdrevet utvikling (SDD) – arbeidskrav**

**Forklaring på SSD:** at en lager en tydelig prompt før du begynner å kode, det handler også om å gjøre det i riktig rekkefølge, begynner fra starten og forklarer steg for steg i prosessen. Da vil du få en mer nøyaktig kode og KI vil ikke lenger gjette seg frem til hva du vil, og du unngår misforståelser. Større prosjekter vil bli enklere å styre og alle i teamet jobber etter samme 'oppskrift'.

### **1. Prosjektets mål**

Utvikle en moderne, KI-støttet turplattform for TiU som gir brukere tilgang til turruter, hytter og turmål, tilbyr kart og navigering, støtter fellesturer (fast og fleksibel dato), har chat og sosial funksjonalitet, egne portaler for hytteeiere og annonsører, admin-panel, flerspråklighet og integrasjon med vær-API.

### **Kjerneprinsipper**

- Spesifikasjoner styrer utviklingen (én sannhetskilde).
- Tydelig intensjon og akseptkriterier før implementering.
- Ingen uklarheter.
- Modulbasert system.
- KI-drevet arbeidsflyt: Plan → Implementering → Validering.
- Brukersentrert design.
- Skalerbar, tilgjengelig og sikker løsning.

### **Roller**

- Vanlig bruker
- Turleder
- Hytteeier
- Annonsør
- Admin

### **Systemmoduler**

1. Kart
2. Turruter
3. Hytter
4. Fellesturer (fast dato)
5. Fellesturer (fleksibel dato)
6. Chat
7. Brukere og roller
8. Favoritter og anmeldelser
9. GPS-navigering
10. Annonseportal
11. Admin-panel
12. Vær-API
13. Flerspråklighet
14. Database + REST API

## **Funksjonelle krav**

### **Kart**

- Vise ruter, hytter og turmål
- Filtrering
- Tegning av ruter
- GPX-import
- Høydedata
- GPS-navigering

### **Turruter**

- Registrering via tegning, GPX eller eksisterende stier
- Metadata
- Kobling til hytter'

## **Hytter**

- Betjent/ubetjent
- Fasiliteter, pris
- Undersider
- Kartplassering

## **Fellesturer – fast dato**

- Påmelding
- Ledige plasser
- Kartvisning
- Gruppechat

## **Fellesturer – fleksibel dato**

- Tidsintervall
- Interessepåmelding
- Hytter melder kapasitet
- Vær-API påvirker dato
- Bindende påmelding
- Låsing av dato
- Romreservasjoner
- Rabatt for tidlig påmelding

## **Chat**

- Direktemeldinger
- Gruppechat
- Bildedeling

## **Brukere og roller**

- Registrering
- Roller
- Profil
- Favoritter og anmeldelser

## **Annonseportal**

- Annonserregistrering
- Godkjenning
- Annonser med bilder/tekst
- Kategorier
- Betaling per visning/klikk
- Statistikk

## **Admin**

- CRUD for alle data
- Godkjenning av turer og annonser

## **Vær-API**

- Vær per turmål og dato

## **Flerspråklighet**

- react-i18next
- Språkfiler

## **Ikke-funksjonelle krav**

- Mobilvennlig
- Responsivt kart
- Sikker autentisering
- Skalerbar backend
- Robust database
- God ytelse på mobil

- Tilgjengelighet (WCAG)

### **Systembegrensninger**

- React frontend
- REST API backend
- react-i18next
- OpenLayers/Leaflet
- Yr-API
- Database
- Ingen mobilapp-rammeverk

### **Akseptansekriterier (kortversjon)**

- Søk og filtrering i kart fungerer
- Påmelding til turer fungerer
- Fleksibel dato låses når nok deltakere
- Hytteeiere kan melde kapasitet
- Annonserer kan opprette annonser og se statistikk
- Chat fungerer med bilder
- GPS-navigering fungerer
- Løsningen er flerspråklig

### **Edge Cases**

- Flere interessepåmeldinger
- Overlappende bindende påmeldinger
- Hytte trekker kapasitet
- Manglende værdata
- Korrupt GPX
- Ugyldig annonsetid

- Mange bilder i chat

### **Avklaringer som trengs**

- Minimum deltakere for låsing
- Regler for rabatt
- Maks bilder i chat
- Språkvalg
- Betalingsmetoder

### **Arkitektur**

- React
- Node/Express eller .NET
- PostgreSQL/MongoDB
- REST API
- OpenLayers/Leaflet
- Yr-API
- JWT
- WebSockets

## **12. Oppgave-nedbrytning**

**Backend:** API, roller, turer, hytter, annonser, fleksible turer, vær, chat, statistikk

**Frontend:** Kart, filtrering, tur-UI, chat, admin, språk

**Database:** Skjemaer, relasjoner, indekser

## **13. Validering**

- Enhetstester
- Integrasjonstester
- UI-tester

- Belastningstester
- API-tester
- Akseptansetester

Hvordan KI og jeg kom frem til dette

Jeg gav KI:

**Prosjektkravene**

**SDD-kilder**

**KI brukte begge deler til å lage en strukturert, profesjonell spesifikasjon for hele turappen.**

## 8. Nettsidedesign med CSS Grid [arbeidskrav]

Et grid-basert layout system deler inn nettsiden i et "rutenett" av rader og kolonner. Så kan man plassere elementer som header, footer, navbar og ulike typer innhold på dette grid-et. Bruk [turappen \(prosjekt 2025/2026\)](#) som eksempel. Lag et forslag til "standard" layout for (noen av) nettsidene ved hjelp av CSS Grid eller tilsvarende løsning i f.eks. [Tailwind](#) , [Bootstrap](#) eller [Material UI](#) . Kanskje du trenger litt [eksempeltekst](#) å sette inn?

Du kan bruke KI for å løse (også) denne oppgaven, men du må forstå og kunne forklare koden som blir generert!

## Leksjon 2. JavaScript: DOM-en og event-håndtering

### 1: Oppdatere DOM-en [arbeidskrav]

Som en introduksjon til neste oppgave: Lag en HTML-side med en tekstboks og en knapp. Koble HTML-siden sammen med en JavaScript-fil. Når brukeren klikker i knappen skal det som er skrevet i tekstboksen bli lagt til som et avsnitt på nettsiden. Deretter: Hvis brukeren klikker på dette avsnittet, skal det fjernes fra nettsiden igjen.

Underveis kan du gjerne prøve ut noen av disse [JavaScript-teknikkene \(w3schools\)](#) .:

HTML-siden har en tekstboks (input), en knapp (button) og en tom div som brukes til å vise avsnittene. I JavaScript-fila hentes disse elementene med getElementBuild, og så legger jeg på en click-event på knappen med addEventListener.

Når det klikkes på knappen, leses teksten som står i tekstboksen. Hvis det faktisk står noe der, lager jeg et nytt <p>-element med document.createElement('p'), setter

textContent til teksten brukeren skrev, og legger på en egen click-event på dette avsnittet som gjør at det fjernes igjen når man klikker på det (med removeChild). Til slutt legges det nye <p>-elementet inn i paragraphContainer med appendChild, og tømmer tekstboksen slik at den er klar for neste tekst.

#### **4. Inndatavalidering i turappen [arbeidskrav]**

Lag ett eller flere HTML-skjemaer til turappen ([prosjekt 2025/2026](#)), og legg på relevant inndatavalidering. Prøv å lage et eksempel som krever JavaScript i tillegg til HTML-validering.

Til slutt: Alle valideringsteknikkene du har brukt i denne leksjonen gjøres på klientsiden. Er det tilstrekkelig? :

Nei, det er ikke nok å bare kontrollere data i nettleseren. Brukeren kan skru av JavaScript eller sende inn data på andre måter enn via skjemaet. Derfor må vi også kontrollere (validere) data på serveren før de blir lagret. Validering i nettleseren er mest for å gi brukeren rask beskjed om feil, mens validering på serveren er viktig for sikkerhet og for å hindre at feil data havner i systemet.

### **Leksjon 3. JavaScript og TypeScript: Funksjoner, objekter og JSON**

#### **5. Visning av JSON hyttedata med TypeScript [arbeidskrav]**

I [turappen \(prosjekt 2025/2026\)](#) er det behov for visning av hytter. Foreslå en JSON-representasjon av hyttedata med et hardkodet eksempel og lag deretter en TypeScript-funksjon for visning av hytter. JSON-dataene bør inneholde både objekter, underobjekter og arrays. Hva er fordeler og ulemper med TypeScript i forhold til JavaScript for en slik oppgave? Hvordan kan koden testes ut?